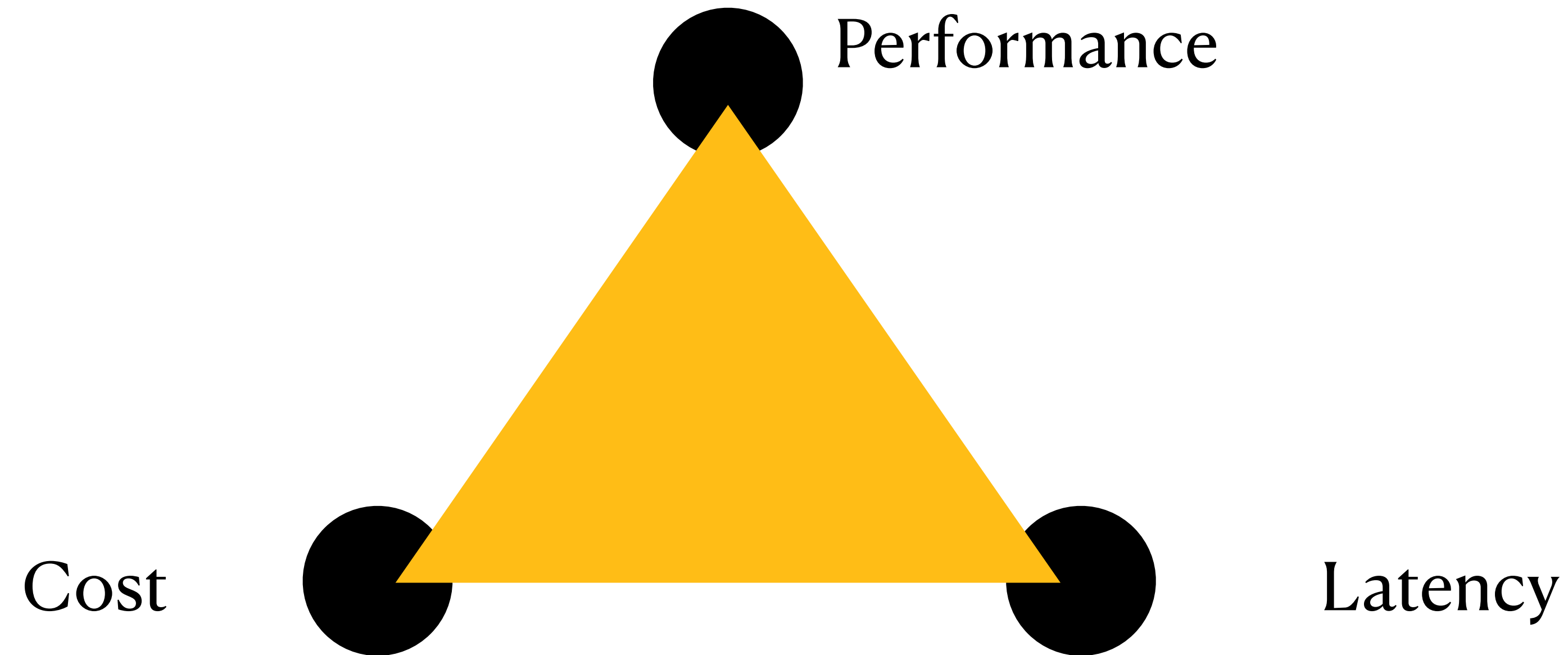


Making your LLM applications run faster and cheaper

Inference Optimization and Caching

Abhijith Neerkaje & Ajay Shenoy

Tradeoffs in building LLM Application



- New models emerge rapidly - but **speed and cost efficiency** remain constant priorities.
- Even the most accurate model is useless if it's **too slow** or **too expensive**.

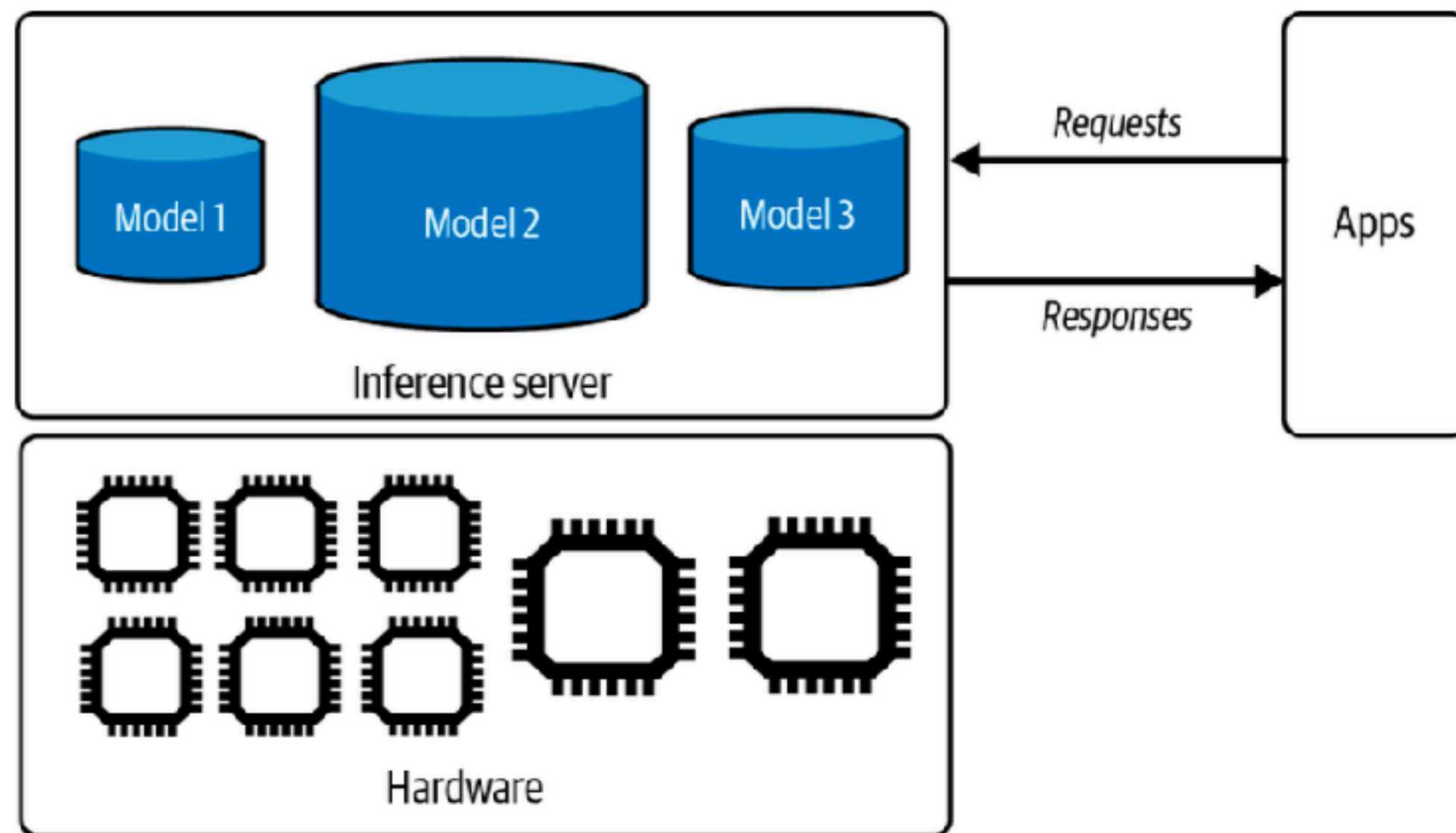
Goal: Improve **latency** and **cost-efficiency** without sacrificing performance.

Let us relook at a Large Language Model



- Models are first trained with lots of data - Training is a process where we tune the parameters of the model.
- Once the models are trained we can deploy and use them in real world.
- The process of using the model in real world is called **inference**
- If you want a faster response from your Agent , your LLM must perform inference faster

A simple inference service



- The inference server is the core component that runs model inference in production, hosting multiple models ready to serve requests.
- It connects to underlying hardware resources (e.g., GPUs, CPUs) to efficiently execute model computations.
- Applications send inference requests (such as user prompts) to the server, which processes them and returns responses.
- The inference server operates within a broader inference service, responsible for receiving, routing, and preprocessing requests before execution.

The economics of inference

Inference cost is the total expense incurred every time your model generates an output for a user query. It's the variable cost per request, driven primarily by:

- Model size and complexity
- Prompt and output length (token usage)
- Hardware used (GPU, CPU, TPU, etc.)
- Serving infrastructure (batching, caching, scaling)

Cost Structure of LLM Application

Component	Example	% of Total Cost (Approx.)
Model Inference (API or GPU time)	OpenAI API / Self-hosted GPU	80–90%
Storage (Embeddings, logs, cache)	Vector DB, Redis, S3	5–10%
Networking / Serving Layer	Load balancer, serverless calls	3–5%
Monitoring / Retries / Overhead	Logging, observability tools	2–5%

Computing Inference Cost

Approximate per query cost = $1.25 \times (\text{Tokens prompted} \times \text{Cost per input token} + \text{Tokens generated} \times \text{Cost per output token})$



GPT-4 Turbo:

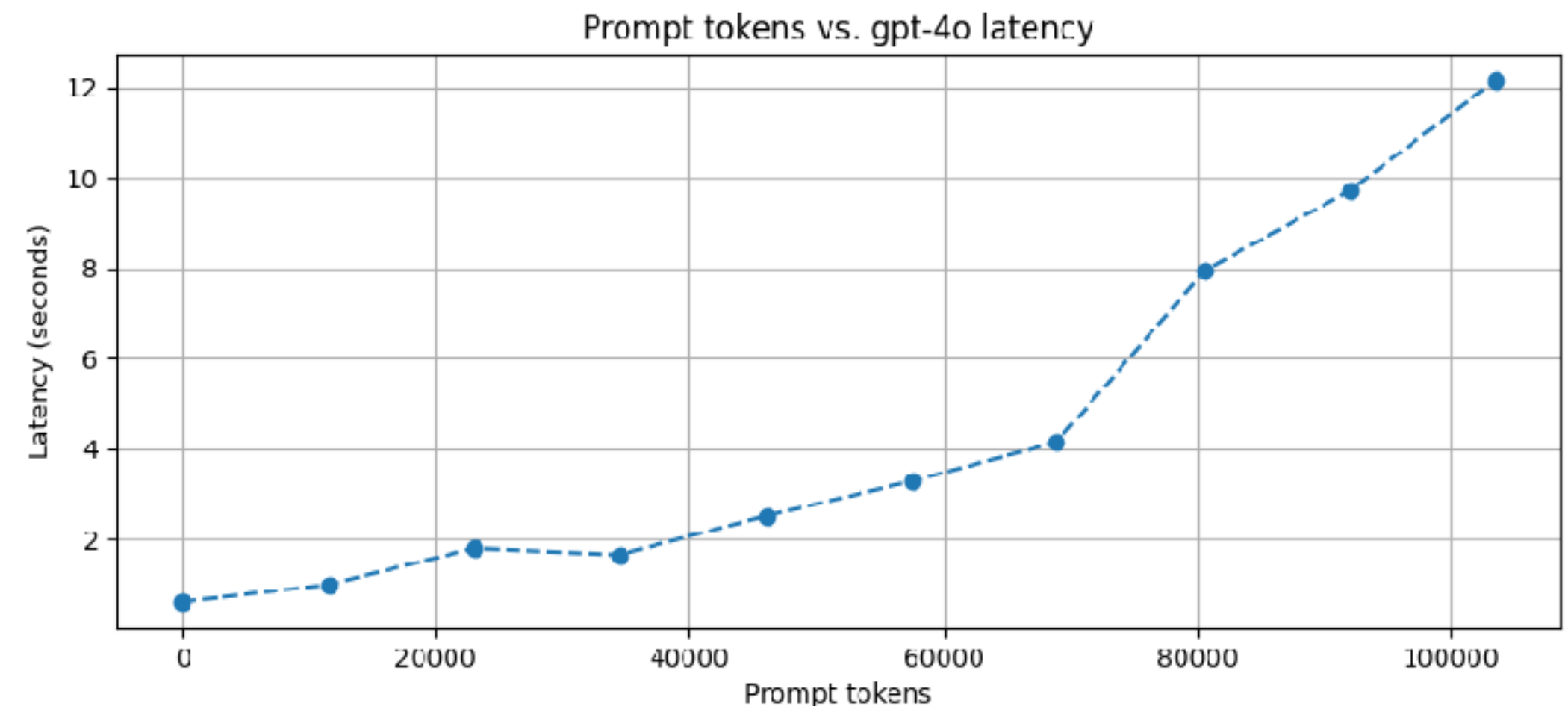
- Input: \$0.01 / 1K tokens
- Output: \$0.03 / 1K tokens
- A 3K prompt + 1K response = \$0.06/query

At Scale:

$10\text{K users/day} \times 5 \text{ queries each} \times \$0.06 \times 1.25 = \$3,750/\text{day} \rightarrow \sim \112 K/month

Latency

- **Latency** = Time from user request → model output.
- Higher latency = lower user satisfaction, retention, and throughput.
- Both Latency and cost scale quadratically with input costs. [Doubling the prompt length (e.g., from 1K to 2K tokens) doesn't double the time/cost—it *quadruples* it.]



Latency - Metrics

- **Time-to-First-Token (TTFT):** How long before the first word appears? Critical for perception.
- **Time-per-Token (TPOT):** How fast does the rest of the text stream?
- **Time -between-Tokens (TBT):** Measure of time between tokens

Total Latency of a query = $TTFT + (TPOT + TBT) \times \text{Number of output tokens}$

Example: $TTFT = 1.2 \text{ Sec}$ $TPOT = 25 \text{ ms/token}$ $TBT = 5 \text{ ms/token}$ Num of output token = 150

Total Latency = $1.2 + 0.03 \times 150 = 1.2 + 4.5 = 5.7 \text{ seconds}$

Things to keep in mind about latency

User's perspective of your application could be different from model performance!

Example:

- Generate a plan, which consists of a sequence of actions. This plan isn't shown to the user.
- Take actions and log their outputs. These outputs aren't shown to the user.
- Based on these outputs, generate a final response to show the user.

Time to publish != Total Latency

Averages can be misleading.

Example:

- Response times for 5 queries : 2s , 3s, 4s, 20s, 3s.
Average latency = 6.4s

More helpful to look at latency in percentiles

Latency Throughput Tradeoff

- **Throughput** :Throughput measures the number of output tokens per second an inference service can generate across all users and requests.
- **Metrics used**: completed requests per minute (**RPM**) or (**Tokens/s/user**)
- Higher throughput implies lower costs
- Techniques like batching can improve throughput but reduce latency.
- Some teams also measure a metric called good put which is number of requests per minute that meets the software level objective.

Inference Optimization

Optimization = identifying bottlenecks and addressing them.

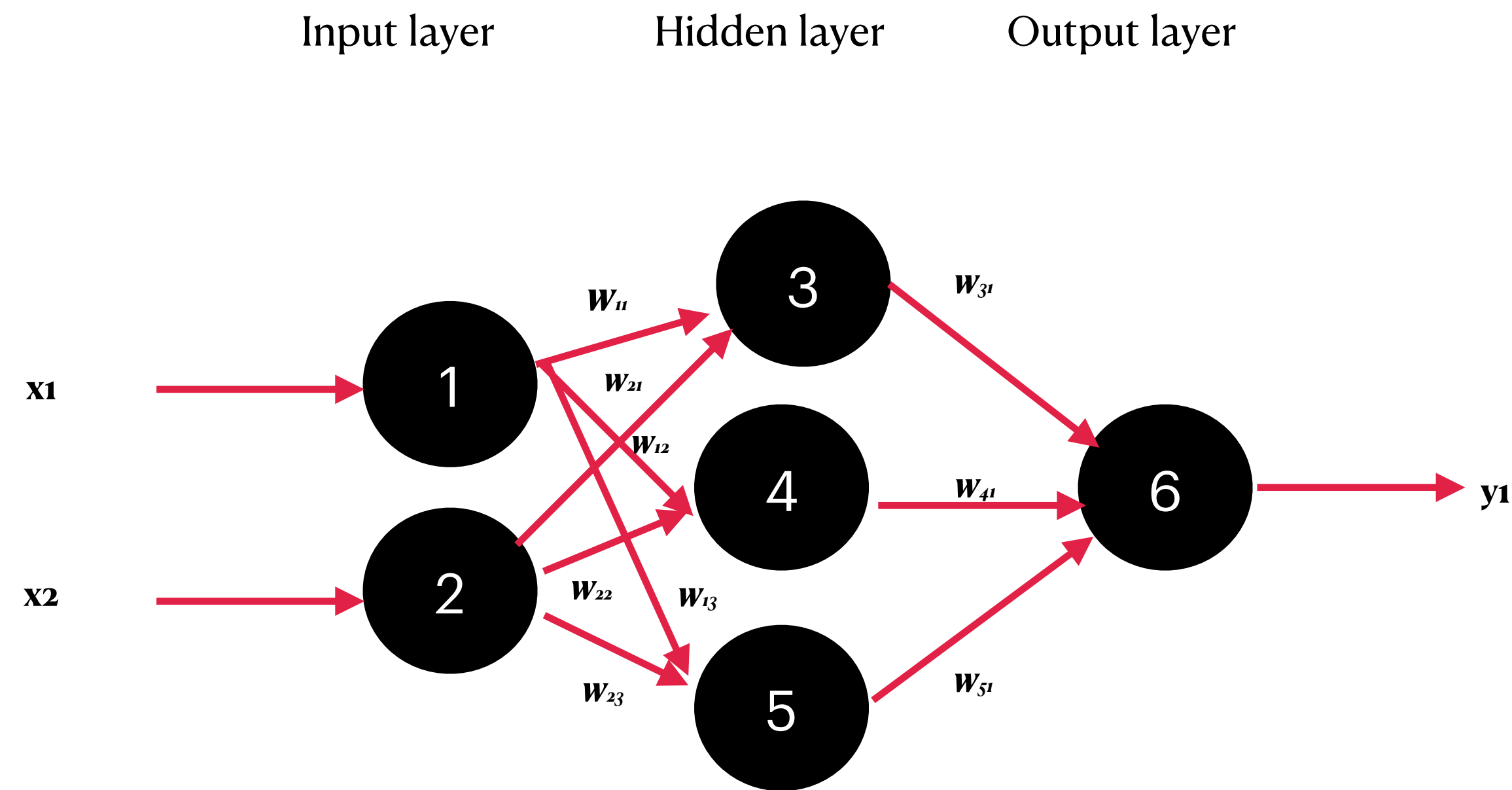
Inference Optimization happens at multiple levels:

- Model Level – Reduce model size or use efficient architectures (e.g., optimized attention).
- Hardware Level – Deploy on optimized or specialized compute hardware.
- Service Level – Use caching, batching, and smart resource allocation to cut latency & cost.

Model Level Inference Optimization

- Transformer based models are resource intensive because of
 - Model size
 - Attention mechanism
 - Autoregressive nature of decoding
- Model Level Optimization– Reduce model size or use efficient architectures (e.g., optimized attention).

Quantization



- Technique to reduce model precision (e.g., from FP32 $\rightarrow$ INT8 or INT4)
- Converts large floating-point weights into lower-bit representations to save memory and computation

Goal	Impact
Smaller model size	4 $\times$ –8 $\times$ reduction in memory footprint
Faster inference	Up to 2–4 $\times$ speed-up (less data movement)
Lower cost	Cheaper GPU/CPU requirements
Minimal accuracy loss	Usually <1% drop if done correctly

Model Distillation

What is Distillation?

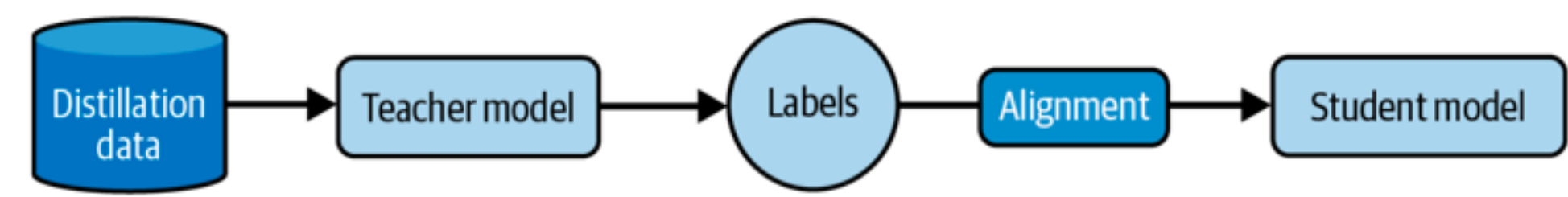
Model distillation is the process of transferring knowledge from a large, accurate model (**Teacher**) to a smaller, faster model (**Student**) that mimics its behavior.

Goal:

Achieve comparable accuracy with faster inference and lower cost.

How It Works:

- Train a large Teacher model to produce soft labels (probability distributions).
- Train a smaller Student model to replicate the teacher’s outputs.
- Deploy the Student model for inference — smaller size means faster response and lower compute cost.



Metric	Teacher (e.g., GPT-J 6B)	Student (e.g., DistilGPT2)	Improvement
Model Size	24 GB	3 GB	~8× smaller
Inference Latency	1.0×	0.4×	60% faster
Cost per Query	1.0×	0.3×	70% cheaper

Model Pruning

What is Pruning?

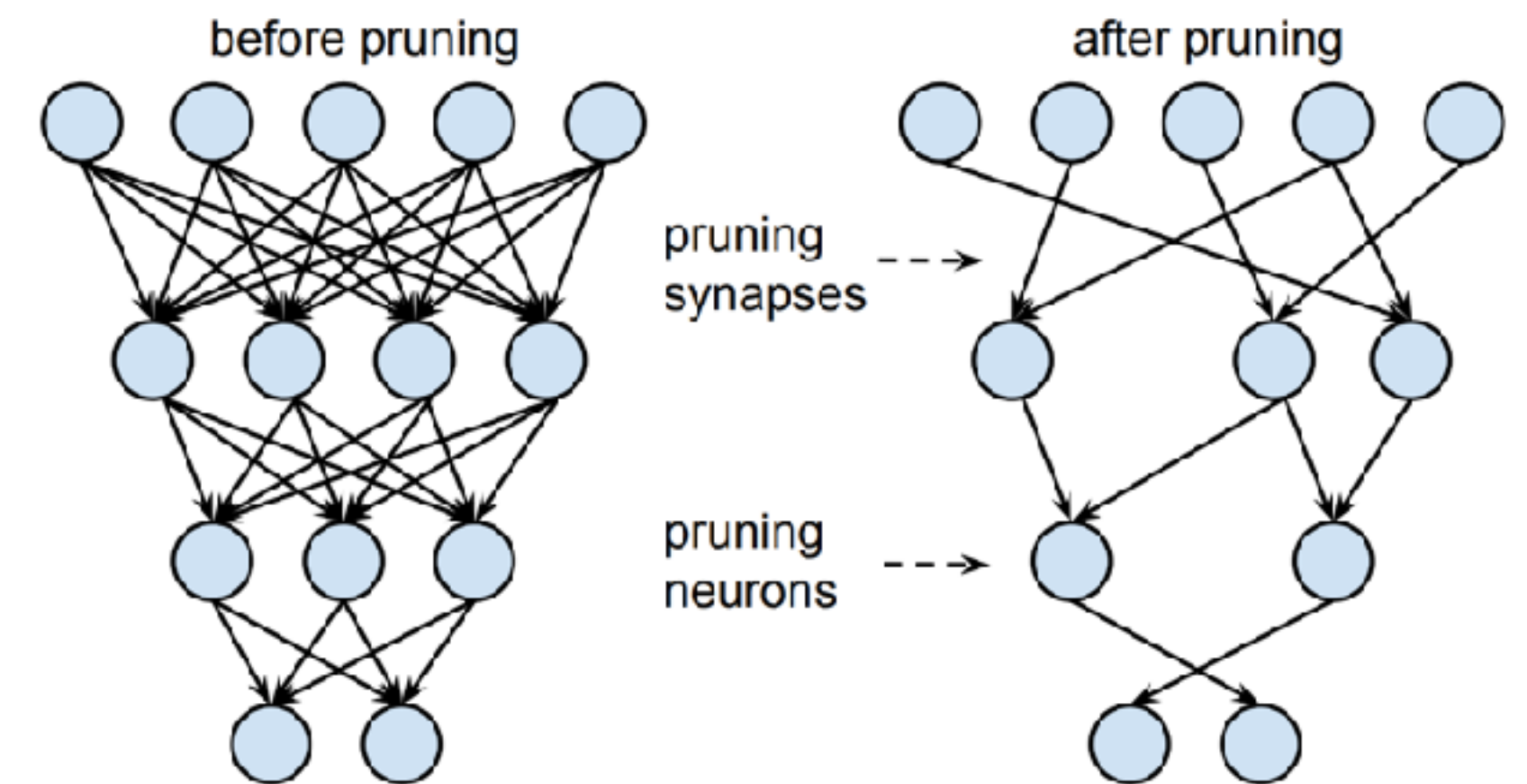
Model pruning eliminates unnecessary parameters (weights or neurons) from a trained model while preserving most of its accuracy.

The result is a smaller, faster network with reduced memory and compute requirements.

How It Works:

- Train a full model until convergence.
- Identify unimportant weights — those contributing little to output.
- Remove or zero them out.
- Fine-tune the pruned model to recover lost accuracy.

Risky approach susceptible to catastrophic forgetting!



Optimized Serving Frameworks/Runtimes

- Specialized systems to run LLMs efficiently using hardware acceleration, memory optimization, and batching — reducing latency and cost.
- Standard PyTorch/TensorFlow inference is not production-optimized.
- Optimized runtimes use kernel fusion, graph optimization, and GPU/TPU/NPU acceleration.
- Can achieve 2–5× speedups and 40–70% cost savings at scale.
- Relevant only if you are self hosting the llm

Framework	Use Case
ONNX Runtime	Edge & cloud inference with quantization & graph fusion
TensorRT	GPU-based LLM serving (FP16/INT8)
vLLM	Chat & streaming responses with PagedAttention
Triton Inference Server	Production-scale deployment (multi-framework)

Solving autoregressive nature of llm: Speculative decoding

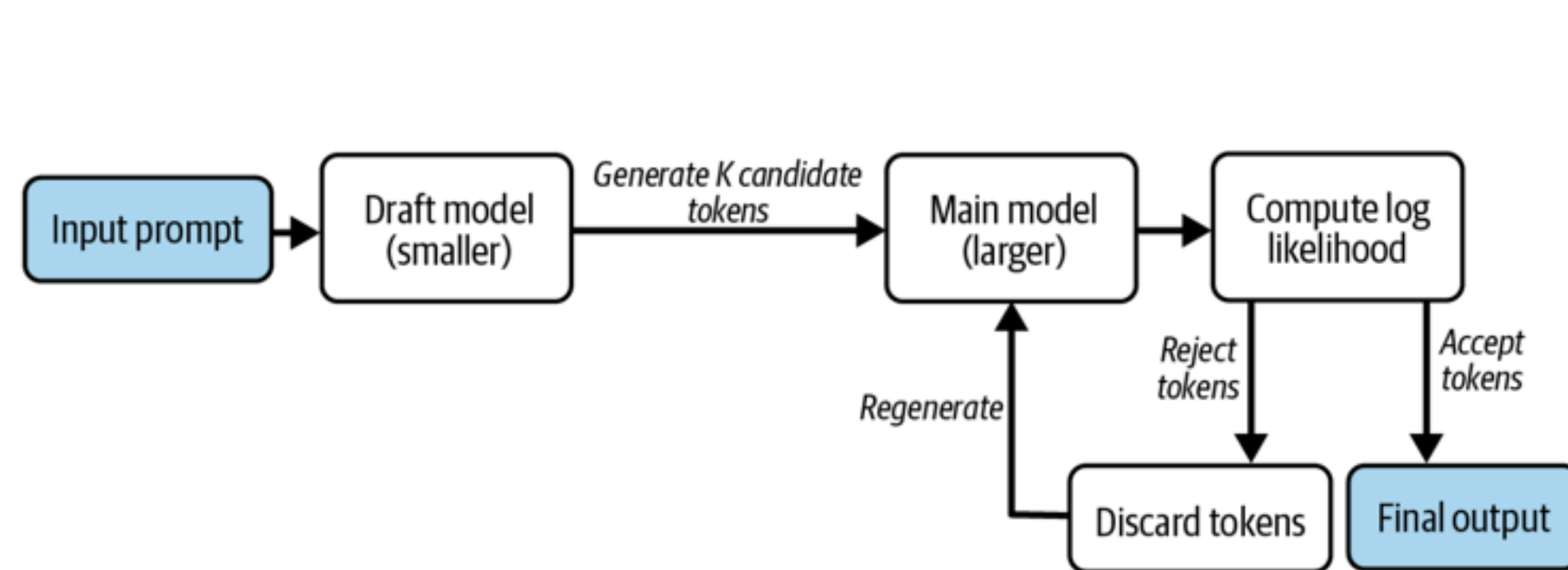


Image credit: Suhas Pai

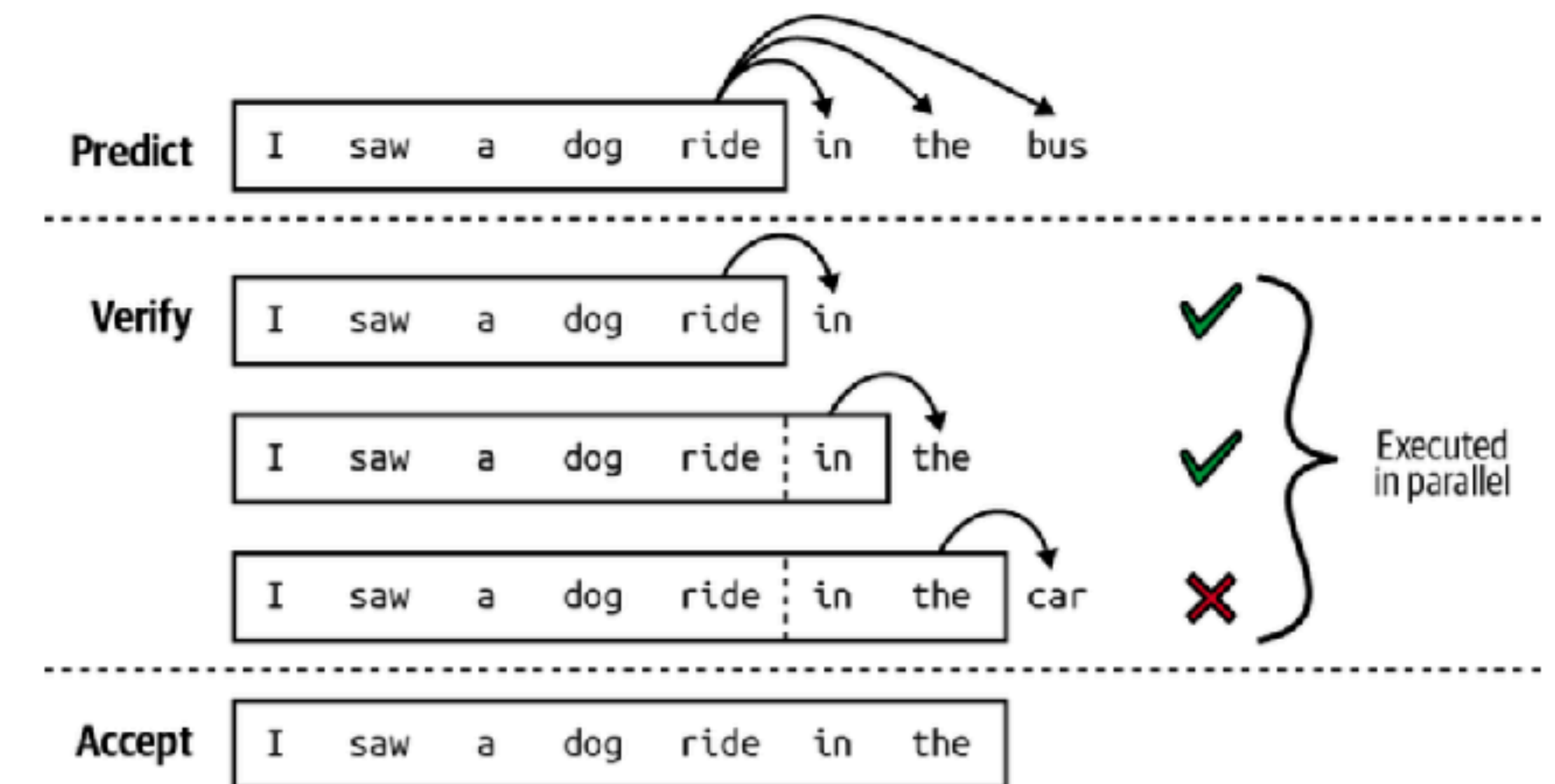
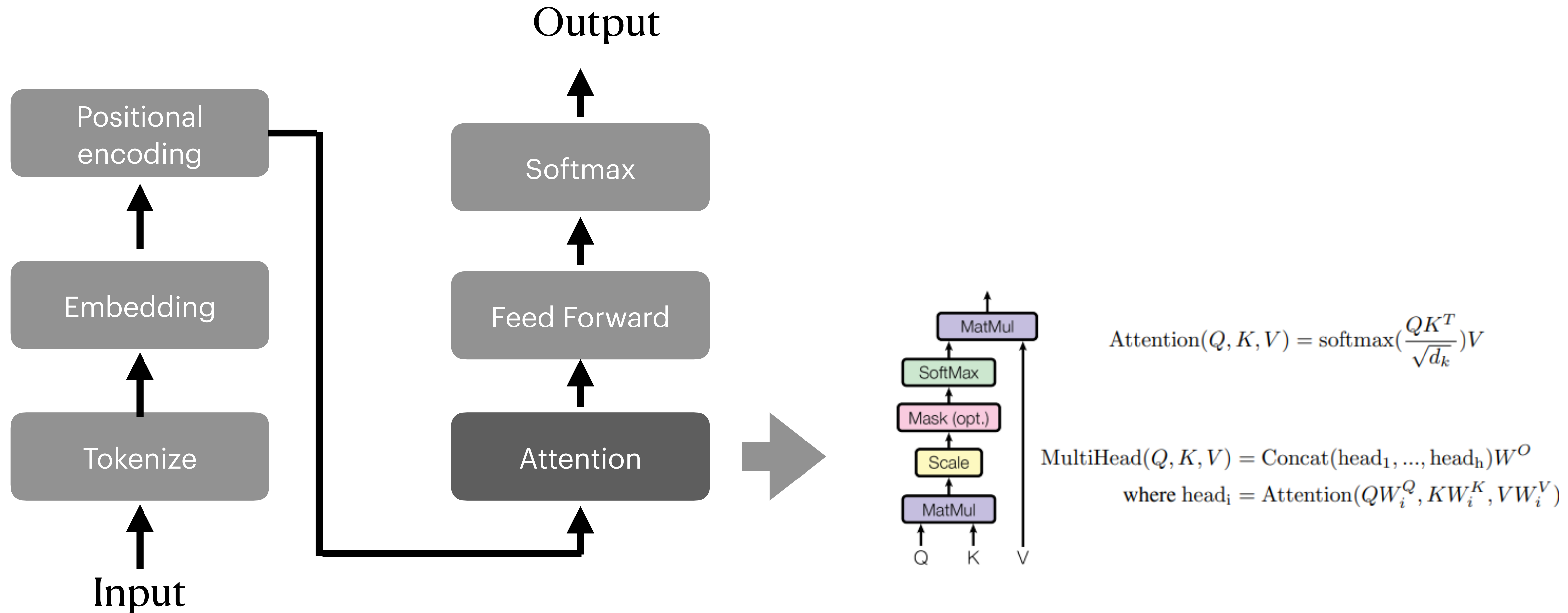


Image credit: Chip Hyuen

- A **smaller draft model** quickly generates multiple candidate tokens, which are then **verified by a larger main model** for accuracy.
- The **main model** computes the likelihood of each candidate token and **accepts or rejects** them based on confidence.
- This approach enables **parallel verification** of multiple tokens, significantly **speeding up inference** without sacrificing output quality.

Digression: Transformer Architecture



KV Caching

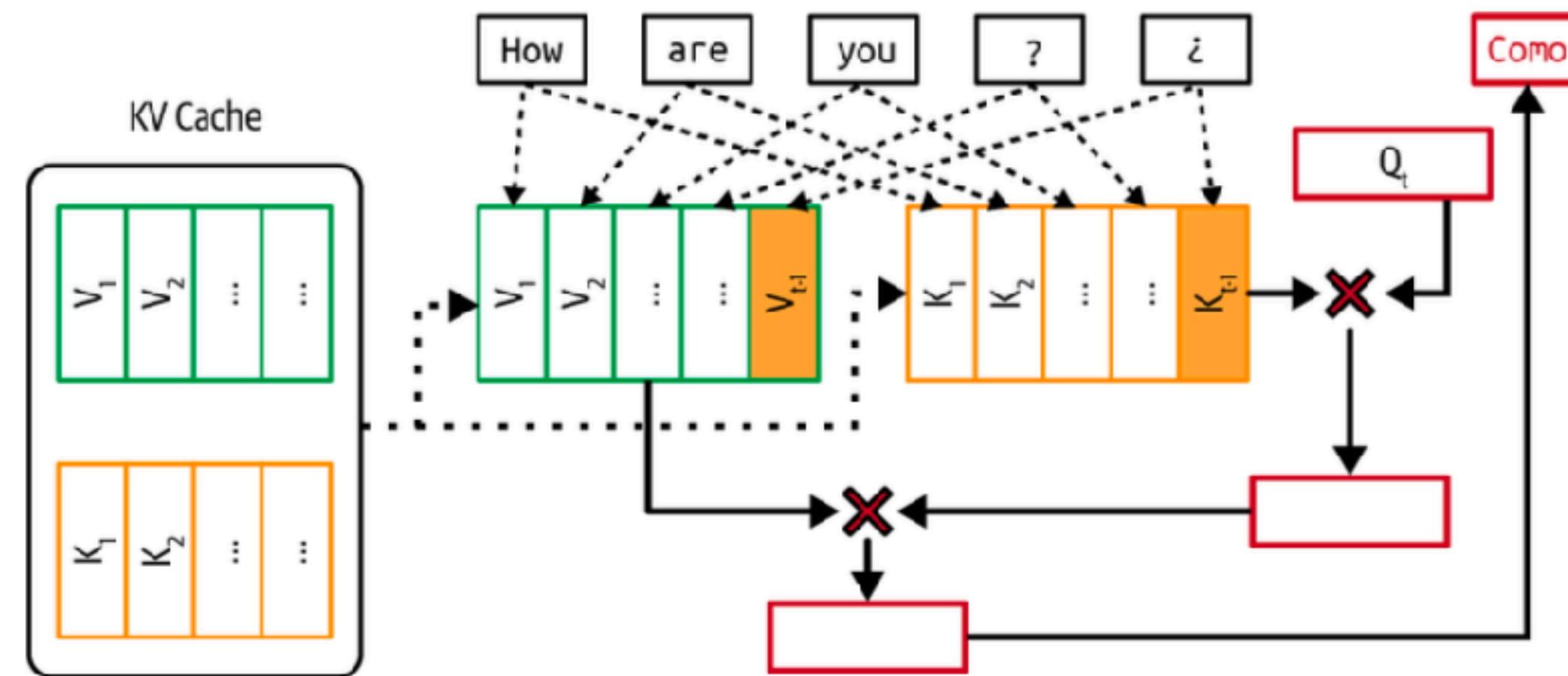


Image credit: Chip Hyuen

Standard Approach

- To generate token t , the model reprocesses the entire sequence up to t .
- At each layer, Key and Value vectors for tokens $1 \dots t-1$ are recomputed.
- This repeated computation is inefficient, as the K and V vectors for earlier tokens do not change.

KV Caching Approach

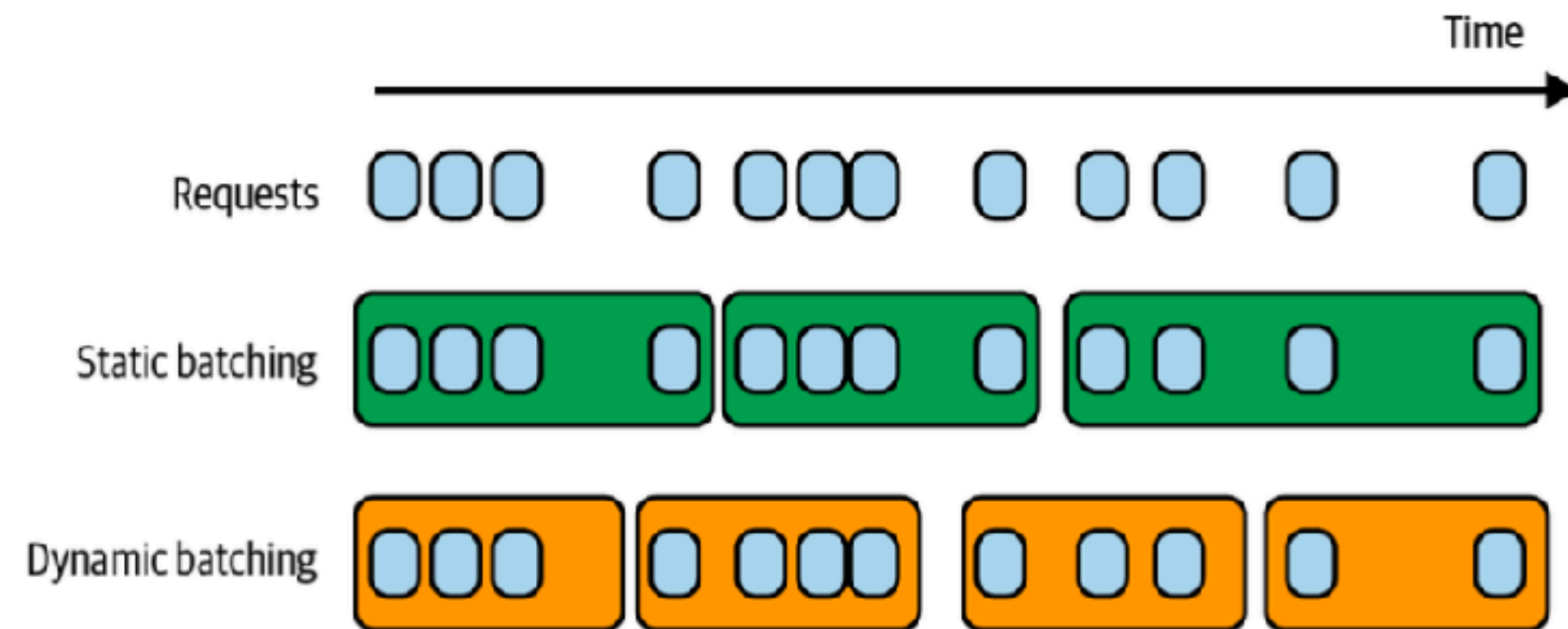
Cache K and V vectors for all previous tokens at every layer to avoid re-computation.

Step 1 – Prefill: Process the full input prompt once and store K, V matrices for each layer.

Step 2 – Generate: For each new token, compute its K, V , then append them to the cache.

The Attention mechanism uses cached context to generate the next output efficiently.

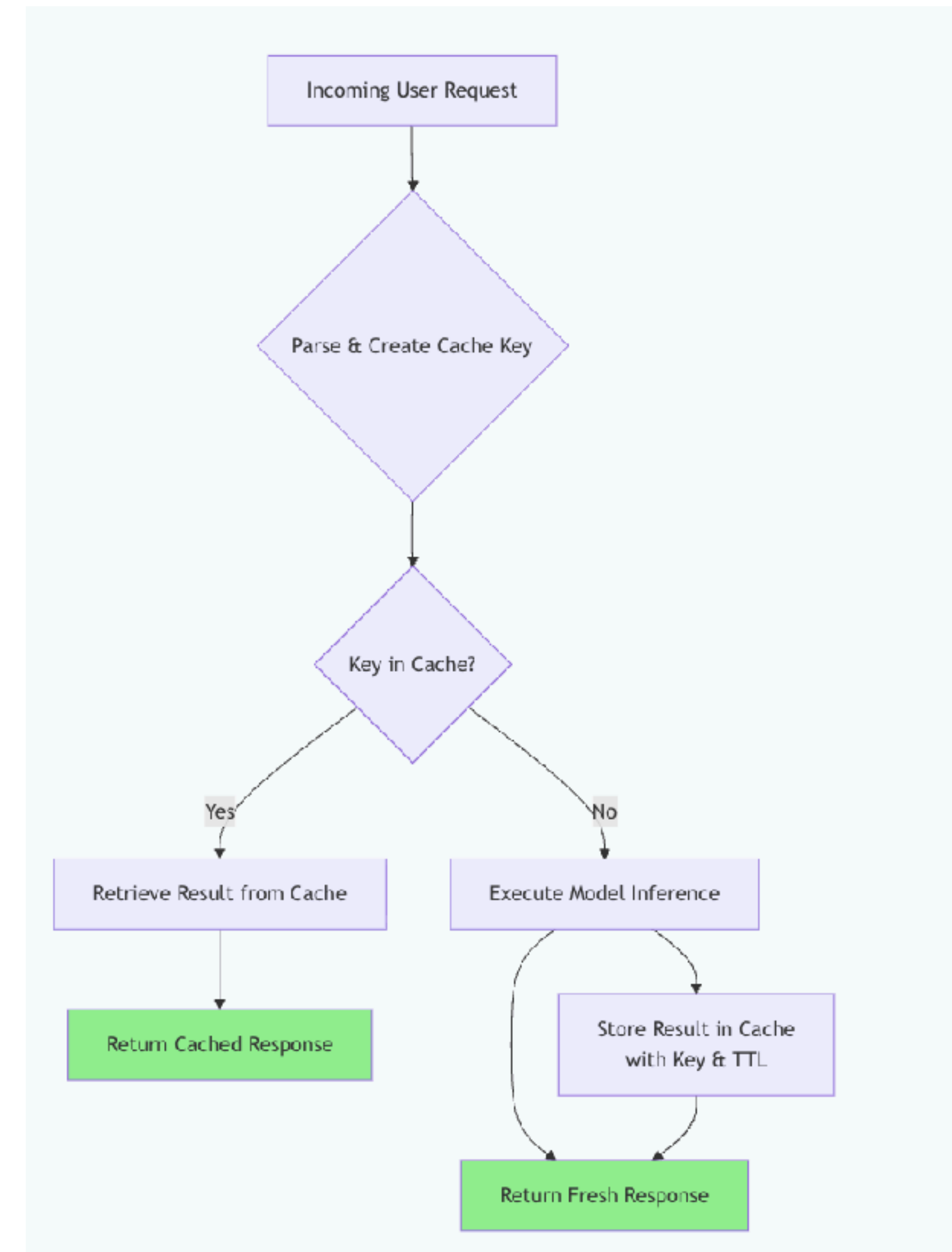
Service Level Optimization: Dynamic Batching



- **What:** Groups multiple incoming requests into a single batch for parallel processing.
- **Impact:** Maximizes GPU utilization increasing throughput.
- **Trade-off:** Introduces slight latency for batching
- **Workloads suitable for batching**
 - Synthetic data generation
 - Periodic reporting, such as summarizing Slack messages,
 - Sentiment analysis of brand mentions on social media, and
 - Analyzing customer support tickets
 - Onboarding new customers who require processing of all their uploaded documents

Service Level Optimization: Response Caching

- **What:** Stores and reuses the results of identical or similar prompts.
- **Impact:** Eliminates redundant computation for frequent requests, slashing latency & cost to near-zero.
- **Use Case:** Ideal for common queries, session data, or template-based prompts.



Key Takeaways

Core Tradeoffs

- Balance performance, cost, and latency
- Inference costs typically account for 80-90% of total LLM application costs
- Both latency and cost scale quadratically with input length

Key Metrics

- Time-to-First-Token (TTFT): Critical for user perception
- Time-per-Token (TPOT): Streaming speed
- Throughput: Output tokens per second across all users
- Look at percentiles, not just averages

Key Takeaways : A Multi-Level Optimization Strategy

Level	Key Techniques	Impact
Model	Quantization, Distillation, Pruning	2-8x smaller models, 60-70% faster inference
Hardware	Optimized Runtimes (vLLM, TensorRT)	2-5x speedups, 40-70% cost savings via kernel fusion & graph optimization
Algorithm	KV Caching , Speculative Decoding	Eliminates redundant computation, drastically reduces latency per token
Service level optimization	Dynamic batching, response caching	

Production performance requires a holistic approach, combining efficient models, specialized hardware, and intelligent serving strategies.